

# C++ | CPL

## Summary

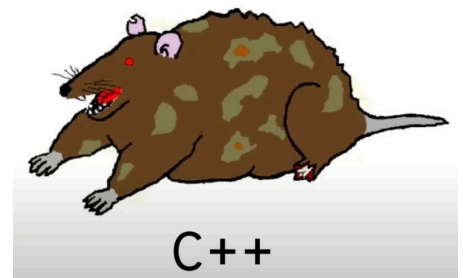
---

## CONTENTS

|                                          |          |
|------------------------------------------|----------|
| <b>1. Why C++?</b>                       | <b>2</b> |
| <b>2. Intro</b>                          | <b>2</b> |
| <b>2.1. Compilation</b>                  | <b>2</b> |
| <b>2.2. Declarations and definitions</b> | <b>3</b> |
| 2.2.1. #include guards                   | 3        |
| 2.2.2. Functions                         | 3        |
| 2.2.3. Values and references             | 3        |
| <b>2.3. Libraries</b>                    | <b>4</b> |
| <b>3. Values and Streams</b>             | <b>4</b> |
| <b>3.1. Variables</b>                    | <b>4</b> |
| <b>3.2. Literal Values</b>               | <b>4</b> |
| <b>3.3. Expressions</b>                  | <b>5</b> |
| <b>3.4. Type Conversion</b>              | <b>5</b> |
| <b>Bibliography</b>                      | <b>6</b> |

## 1. WHY C++?

- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Being called stupid when you think that the move function actually moves things or if you confuse a const pointer with a pointer of const
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Good excuse for alcoholism
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- Headaches (if you're a masochist)
- 
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Ever wanted to have 100 different ways to do the same thing? Now you can
- Has a pretty mascot [\[1\]](#)



## 2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

- Header files can be either `*.hpp` or `*.h`
- Regarding include order: It is good practice to include own-headers ("`"`") before system-headers ("`<`"). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes.***

| Do's | Don'ts                                                 |
|------|--------------------------------------------------------|
|      | Using global variables                                 |
|      | Redundant returns that would be implicit               |
|      | <code>using namespace</code>                           |
|      | Not initializing a variable upon definition            |
|      | Not using <code>const</code> enough                    |
|      | <b>TODO:</b><br>make this column real big for the lulz |

### 2.1. COMPILATION

3 Phases of compilation:

**Preprocessor:** Textual replacement of preprocessor directives (`#include`)

**Compiler:** Translation of C++ code into machine code (source file to object file)

**Linker:** Combination of object files and libraries into libraries and executables

## 2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (**One Definition Rule**/ODR), in contrast to declaration, which can be done several times.

### 2.2.1. `#include` guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

### 2.2.2. Functions

Declarations

```
auto <function-name>(<parameters>) → <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto <function-name>(<parameters>) → <return-type> { /*body*/ }
<return-type> <function-name>(<parameters>) { /*body*/ }
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

### 2.2.3. Values and references

```
struct Point {
    int x;
    int y;
};
```

Copying vs. Sharing:

```
Point point{1, 20};
```

```
Point copiedPoint{point};
```

```
Point &sharedPoint{point};
```

```
auto copydPointParam(Point point) → Point { }
auto referencePointParam(Point & point) → Point { }
```

## 2.3. LIBRARIES

- [catch2](#)
- [boost](#)

**TODO:**

cmake

## 3. VALUES AND STREAMS

### 3.1. VARIABLES

```
<type> <variable-name>{<initial-value>;
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (**constexpr**). The keyword **const** also appears in other contexts.

```
int const theAnswer{42};
double constexpr pi{3.14};
```

You should use **const** whenever possible for non-member variables.

### 3.2. LITERAL VALUES

| <i>Literal Example</i>          | <i>Type</i>        | <i>Value</i>              |
|---------------------------------|--------------------|---------------------------|
| 'a'                             | char               | Letter a, value: 97       |
| '\n'                            | char               | <NL> character, value: 10 |
| '\x0a'                          | char               | <NL> character, value: 10 |
| 1                               | int                | 1                         |
| 42L                             | long               | 42                        |
| 5LL                             | long long          | 5                         |
| int{} // (not really a literal) | int                | 0 (default value)         |
| 1u                              | unsigned int       | 1                         |
| 42uL                            | unsigned long      | 42                        |
| 5uLL                            | unsigned long long | 5                         |
| 020                             | int                | 16 (octal 20)             |
| 0x1f                            | int                | 31 (hex 1F)               |
| 0XFULL                          | unsigned long long | 15 (hex F)                |
| 0.f                             | float              | 0                         |

|                         |                |                                        |
|-------------------------|----------------|----------------------------------------|
| <code>.33</code>        | double         | 0.33                                   |
| <code>1e9</code>        | double         | 1000000000 (10 <sup>9</sup> )          |
| <code>42.E-12L</code>   | long double    | 0.000000000042 (42*10 <sup>-12</sup> ) |
| <code>.3l</code>        | long double    | 0.3                                    |
| <code>"hello"</code>    | char const [6] | Array of 6 chars: h e l l o            |
| <code>"\012\n\\"</code> | char const [4] | Array of 4 chars:                      |

---

### 3.3. EXPRESSIONS

*TODO:*

---

### 3.4. TYPE CONVERSION

*TODO:*

---

## BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sept. 12, 2026. [Online]. Available: [https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink\\_800/feedshare-shrink\\_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC\\_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE](https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE)